Übungsaufgaben – Grundlagen der Programmierung

Python · DHBW Heilbronn · Wirtschaftsinformatik, 1. Semester · Dr. Markus Rosenfelder

Klausur laut Lernplan: Fr, 11.12.2026, 120 Minuten · Stoff: VL1–VL5 (Stand 21.10.2026) · inoffizielles Übungsmaterial, es gibt keine Musterklausur

Zwei Probeklausuren im Papierformat (je 120 Minuten, 120 Punkte) und ein Aufgabenpool je Vorlesung, aufsteigend schwer (★ leicht bis ★★★ anspruchsvoll). Die Lösungen stehen in der separaten Datei `loesungen_programmierung.pdf`.

Empfehlung: Erst den Pool der jeweiligen Vorlesung, dann eine Probeklausur am Stück unter Klausurbedingungen (ohne Rechner, 120 Minuten), danach mit dem Erwartungshorizont selbst korrigieren. Die zweite Probeklausur einige Tage später.

Inhalt

1. Probeklausur A
2. Probeklausur B
3. Aufgabenpool VL1: Einführung: Algorithmen, Datenstrukturen, Python-Werkzeuge (6 Aufgaben)
4. Aufgabenpool VL2: Variablen, Datentypen und Operatoren (7 Aufgaben)
5. Aufgabenpool VL3: Kontrollstrukturen und Listen (8 Aufgaben)
6. Aufgabenpool VL4: Funktionen, Rekursion und Tests (8 Aufgaben)
7. Aufgabenpool VL5: Objektorientierte Programmierung I (7 Aufgaben)

> Offen ist, ob die Klausur auf Papier oder am Rechner geschrieben wird. Die Probeklausuren sind als Papierklausur gestaltet; am Rechner lassen sich dieselben Aufgaben im Texteditor lösen und mit `uv run`, `ruff check` und `ty check` prüfen. Hilfsmittel (z. B. ein Spickzettel) sind nicht bekannt – angenommen ist „keine“.

Abgedeckte Themen

- VL1: Algorithmusbegriff und Eigenschaften, Datenstrukturen, Interpreter/Compiler/Zwischencode, Python-Syntax, uv/ruff/ty
- VL2: Datentypen, Variablen, `Final`, Operatoren (`/ // % **`), Strings, f-Strings, Typumwandlung, `Decimal`, Syntax vs. Semantik
- VL3: `if/elif/else`, `match`, bedingter Ausdruck, truthy/falsy, Kurzschluss, `while`, `for`/`range`, `break`/`continue`, Listen, `enumerate`, `zip`, Matrix, Tupel
- VL4: Funktionen, Typannotationen, Docstrings, `main()`, `return`/`None`, Gültigkeit (LEGB, `UnboundLocalError`), Parameterübergabe, Standardwerte, `*args`, Rekursion, `assert`, pytest
- VL5: Klassen, Objekte, `__init__`, `self`, Methoden, `__str__`/`__repr__`, Identität, Klassenattribute, Kapselung, `@property`, Hat-ein-Beziehung, `@dataclass`

Probeklausur A

Grundlagen der Programmierung (Python)

DHBW Heilbronn · Wirtschaftsinformatik · 1. Semester · Dr. Markus Rosenfelder
Übungsklausur zur Vorbereitung auf die Klausur am Fr, 11.12.2026 – **keine offizielle Musterklausur**

Name, Vorname _____ Kurs _____

Matrikelnummer

Hinweise

- Bearbeitungszeit: **120 Minuten**. Erreichbar sind **120 Punkte** (Faustregel: 1 Punkt ≈ 1 Minute).
- Hilfsmittel: keine (Annahme – bitte für die echte Klausur erfragen). Schreiben Sie mit dokumentenechtem Stift, nicht mit Rot.
- Schreiben Sie Python 3.14 mit Typannotationen an Parametern und Rückgabewerten. Machen Sie Einrückungen deutlich sichtbar (4 Leerzeichen pro Ebene).
- Multiple Choice: Pro Frage ist **genau eine** Antwort richtig. Falsche Kreuze geben 0 Punkte, keine Minuspunkte.
- Verwenden Sie nur Sprachmittel aus VL1–VL5. Eingebaute Funktionen wie `max()` oder `str.count()` sind erlaubt, wenn die Aufgabe sie nicht ausdrücklich verbietet.
- Reicht der Platz nicht, schreiben Sie auf die Rückseite und vermerken Sie das bei der Aufgabe.

Aufgabe	Thema	max. P	erreicht
1	Multiple Choice (VL1–VL5)	12	
2	Ausdrücke auswerten (VL2)	10	
3	Was gibt dieser Code aus? (VL3, VL4)	10	
4	Trace-Tabelle (VL3)	10	
5	Fehler finden und korrigieren (VL2–VL4)	14	
6	Lückencode mit match (VL3, VL4)	10	
7	Funktion schreiben (VL4)	12	
8	Rekursion (VL4)	12	
9	Tests mit pytest (VL4)	8	
10	Klassen und Objekte (VL5)	14	
11	Konzeptfragen (VL1, VL4, VL5)	8	
	Summe	**120**	

Aufgabe A1: Multiple Choice — 12 P

Kreuzen Sie jeweils **genau eine** richtige Antwort an (je 2 P).

a. Wie führt CPython die Datei `hallo.py` aus?

- ☐ Ein Compiler erzeugt vorab eine eigenständige Maschinencode-Datei für die Zielplattform.
- ☐ Der Quellcode wird in Bytecode übersetzt, den die Python-VM ausführt.
- ☐ Die CPU führt den Quelltext ohne jede Übersetzung direkt aus.
- ☐ Der Code wird in Java-Bytecode übersetzt und von der JVM ausgeführt.

b. Welchen Wert hat `-7 // 2`?

- ☐ `-3`
- ☐ `-3.5`
- ☐ `-4`
- ☐ `3`

c. Welcher Wert gilt in `if wert:` als *falsch*?

- ☐ `"0"`
- ☐ `[0]`
- ☐ `-1`
- ☐ `0.0`

d. Welche Werte nimmt `i` in `for i in range(2, 11, 3):` an?

- ☐ 2, 5, 8, 11
- ☐ 2, 5, 8
- ☐ 3, 6, 9
- ☐ 2, 4, 6, 8, 10

e. Ein Algorithmus heißt *determiniert*, wenn …

- ☐ er bei jeder erlaubten Eingabe nach endlich vielen Schritten endet.
- ☐ gleiche Eingaben und Startbedingungen immer das gleiche Ergebnis liefern.
- ☐ zu jedem Zeitpunkt der nächste Schritt eindeutig festgelegt ist.
- ☐ er sparsam mit Zeit und Speicher umgeht.

f. `anna` ist ein Objekt der Klasse `Konto`. Welcher Aufruf bedeutet dasselbe wie `anna.einzahlen(50)`?

- ☐ `Konto.einzahlen(50)`
- ☐ `Konto.einzahlen(anna, 50)`
- ☐ `self.einzahlen(50)`
- ☐ `anna.einzahlen(anna, 50)`

Aufgabe A2: Ausdrücke auswerten — 10 P

Geben Sie für jeden Ausdruck den **Wert** und den **Typ** (`int`, `float`, `str`, `bool`) an. Strings schreiben Sie mit Anführungszeichen. (je ½ P für Wert und Typ)

	Ausdruck	**Wert**	**Typ**
a)	`7 / 2`		
b)	`2 + 3 * 4 ** 2`		
c)	`int(9.99)`		
d)	`"Py" * 2 + "!"`		
e)	`"Heilbronn"[1:4]`		
f)	`len("DHBW Heilbronn")`		
g)	`17 % 5 == 2 and not False`		
h)	`True + 1`		
i)	`5 + 2.0`		
j)	`"30" == 30`		

Aufgabe A3: Was gibt dieser Code aus? 10 P

a) Schleife mit `continue` und `break` (4 P)

```
for i in range(1, 10):
    if i % 3 == 0:
        continue
    if i == 8:
        break
    print(i, end=" ")
print("Ende")
```

Ausgabe:

---

b) Parameterübergabe (6 P)

```
def bearbeite(zahl: int, liste: list[int]) -> None:
    zahl = zahl + 1
    liste[0] = 99
    liste.append(4)
    liste = [7, 7, 7]
    liste.append(5)


def main() -> None:
    zahl = 5
    werte = [1, 2, 3]
    bearbeite(zahl, werte)
    print(zahl, werte)


if __name__ == "__main__":
    main()
```

Ausgabe (3 P) und Begründung in ein bis zwei Sätzen, warum sich `werte` ändert, `zahl` aber nicht (3 P):

Aufgabe A4: Trace-Tabelle — 10 P

```
1  zahl = 4271
2  summe = 0
3  stellen = 0
4  while zahl > 0:
5      summe += zahl % 10
6      zahl //= 10
7      stellen += 1
8  print(f"{stellen} Stellen, Quersumme {summe}")
```

a) Verfolgen Sie den Ablauf. Tragen Sie für jeden Schleifendurchlauf die Werte **am Ende** des Durchlaufs ein. Nicht benötigte Zeilen bleiben leer. (6 P)

Durchlauf	**`zahl % 10` (Z. 5)**	**`summe`**	**`zahl`**	**`stellen`**	**`zahl > 0`?**
Start	–	0	4271	0	

b) Was gibt Zeile 8 aus? (1 P)

c) Was gibt das Programm aus, wenn in Zeile 1 `zahl = 0` steht? Begründen Sie mit der Arbeitsweise von `while`. (3 P)

Aufgabe A5: Fehler finden und korrigieren **14 P**

Das Programm soll den Rabatt einer Bestellung bestimmen (ab 50 Euro 5 %, ab 100 Euro 10 %), den Rabatt ausgeben, die Bestellung im Zähler `bestellungen_gesamt` mitzählen und bei Beträgen über 100 Euro „Großbestellung“ melden. Es enthält **genau vier Fehler** (Syntax-, Laufzeit- und Logikfehler).

```
 1  from typing import Final
 2
 3  RABATT_GRENZE: Final = 100
 4  bestellungen_gesamt = 0
 5
 6
 7  def rabatt_prozent(betrag: float) -> int:
 8      if betrag >= 50:
 9          return 5
10      elif betrag >= 100:
11          return 10
12      return 0
13
14
15  def zaehle_bestellung() -> None:
16      bestellungen_gesamt += 1
17
18
19  def main() -> None:
20      betrag = 120.0
21      prozent = rabatt_prozent(betrag)
22      print("Rabatt: " + prozent + " %")
23      zaehle_bestellung()
24      if betrag > RABATT_GRENZE
25          print("Großbestellung")
26
27
28  if __name__ == "__main__":
29      main()
```

a) Tragen Sie jeden Fehler ein: Zeile, Fehlerart (Syntax / Laufzeit mit Name der Exception / Logik) und eine Korrektur. (je 3 P)

Nr.	Zeile	Fehlerart	Korrektur (Code)
1			
2			
3			
4			

b) Sie starten das unveränderte Programm mit `uv run bestellung.py`. Welcher Fehler wird zuerst gemeldet, und welcher der vier Fehler wird nie durch eine Fehlermeldung angezeigt? (2 P)

Aufgabe A6: Lückencode mit `match` — 10 P

Die Funktion `versandkosten` liefert für eine Versandzone die Kosten oder `None`, wenn nicht geliefert wird. Ergänzen Sie die Lücken (je 1 P).

```
def versandkosten(zone: str, gewicht_kg: float) -> (1)_____:
    (2)_____ zone:
        case "DE":
            kosten = 4.90
        (3)_____:                       # "EU" oder "CH"
            kosten = 9.90
        (4)_____:                       # alle anderen Zonen
            return None
    if gewicht_kg > 5:
        kosten += 1.50
    return kosten


def main() -> None:
    zonen = ["DE", "CH", "US"]
    gewichte = [2.0, 7.5, 1.0]
    for zone, gewicht in (5)_____:     # beide Listen im Gleichschritt
        preis = versandkosten(zone, gewicht)
        if (6)_____:
            print(f"{zone}: nicht lieferbar")
        else:
            print(f"{zone}: {(7)_____} Euro")   # genau zwei Nachkommastellen


if __name__ == "__main__":
    main()
```

Lücke	Ihr Code
(1)	
(2)	
(3)	
(4)	
(5)	
(6)	
(7)	

Was gibt `main()` aus? (3 P)

Aufgabe A7: Funktion schreiben **12 P**

Schreiben Sie eine Funktion `bestellwert` mit Typannotationen und einem Docstring im Google-Stil (Kurzbeschreibung, `Args:`, `Returns:`).

- Parameter: `preise` (Liste von Stückpreisen als `float`), `mengen` (Liste von `int`, gleich lang wie `preise`) und `rabatt` (Anteil als `float`, Standardwert `0.0`).
- Die Funktion summiert Preis × Menge über alle Positionen und zieht den Rabatt ab: `summe * (1 - rabatt)`.
- Ein Rabatt unter 0 oder über 0.5 ist ungültig; dann wird ohne Rabatt gerechnet.
- Das Ergebnis wird mit `round` auf zwei Nachkommastellen gerundet zurückgegeben.

Beispiele: `bestellwert([2.5, 10.0], [4, 3])` → `40.0` · `bestellwert([2.5, 10.0], [4, 3], 0.1)` → `36.0` · `bestellwert([2.5, 10.0], [4, 3], 0.8)` → `40.0`

a) Funktion (10 P)

b) Schreiben Sie den Aufruf für Preise 2.5 und 10.0, Mengen 4 und 3 und 10 % Rabatt nur mit **Schlüsselwortargumenten**, und zwar in der Reihenfolge `mengen`, `preise`, `rabatt`. (2 P)

Aufgabe A8: Rekursion — 12 P

a) Rekursion nachvollziehen (4 P)

```
def r(n: int) -> int:
    if n <= 1:
        return 1
    return n + 2 * r(n - 2)
```

Berechnen Sie `r(5)` und `r(6)`. Schreiben Sie die Aufrufkette mit auf (Abstieg und Aufstieg).

b) Rekursive Funktion schreiben (6 P)

Schreiben Sie eine **rekursive** Funktion `quersumme(n: int) -> int`, die die Quersumme einer nicht negativen ganzen Zahl berechnet, z. B. `quersumme(4271) == 14`. Verwenden Sie keine Schleife und kein `str()`. Benennen Sie im Code (als Kommentar) die Abbruchbedingung und den rekursiven Aufruf.

c) Fehlende Abbruchbedingung (2 P)

Gegeben ist `fakultaet` mit der Abbruchbedingung `if n == 0: return 1` und dem Aufruf `n * fakultaet(n - 1)`. Was passiert bei `fakultaet(-1)`? Wie machen Sie die Funktion robuster?

Aufgabe A9: Tests mit pytest — 8 P

Die Funktion `bestellwert` aus Aufgabe A7 steht in der Datei `bestellung.py`.

a) Schreiben Sie eine vollständige Testdatei für pytest mit mindestens **drei** Testfunktionen, darunter ein Randfall (z. B. leere Bestellung oder ungültiger Rabatt). Geben Sie auch den Dateinamen an. (6 P)

Dateiname: _____

b) Mit welchem Befehl starten Sie die Tests? Was bedeutet die Ausgabezeile `test_bestellung.py .F.`? (2 P)

Aufgabe A10: Klassen und Objekte **14 P**

a) Klasse schreiben (10 P)

Schreiben Sie eine Klasse `Artikel` (Beträge als `Decimal`, `from decimal import Decimal` dürfen Sie voraussetzen):

- Klassenattribut `anzahl`, das jeden neuen Artikel mitzählt; jeder Artikel erhält daraus in `__init__` eine Nummer `A001`, `A002`, … im Attribut `nummer`.
- `__init__(self, name: str, preis: Decimal, bestand: int = 0)` speichert `name` und `bestand` als öffentliche Attribute.
- Der Preis liegt intern in `_preis`. Eine Property `preis` liefert ihn; ihr Setter weist negative Preise mit einer Meldung ab (der alte Preis bleibt). Auch `__init__` benutzt den Setter.
- `verkaufen(self, menge: int) -> bool` verringert den Bestand und liefert `True`. Ist die Menge nicht positiv oder größer als der Bestand, gibt die Methode eine Meldung aus und liefert `False`.
- `__str__` liefert z. B. `A001 Kaffee 1 kg: 12.99 Euro, Bestand 40`.

b) Klassenattribute und Namen (4 P)

Was gibt dieser Code aus?

```
class Zaehler:
    stand = 0

    def __init__(self, name: str) -> None:
        self.name = name
        Zaehler.stand += 1


a = Zaehler("a")
b = Zaehler("b")
c = b
c.name = "c"
b.stand += 10
print(a.name, b.name, c.name)
print(Zaehler.stand, a.stand, b.stand)
print(b is c, a == Zaehler("a"))
```

Aufgabe A11: Konzeptfragen — 8 P

Antworten Sie jeweils in ein bis zwei Sätzen (je 2 P).

a) Was ist der Unterschied zwischen einem *Parameter* und einem *Argument*? Nennen Sie ein Beispiel.

b) Nennen Sie je einen Vorteil und einen Nachteil der Rekursion gegenüber einer Schleife.

c) Was bedeutet *Kapselung*, und mit welchen zwei Mitteln setzt Python sie um?

d) Warum steht der Aufruf `main()` hinter `if __name__ == "__main__":`?

Probeklausur B

Grundlagen der Programmierung (Python)

DHBW Heilbronn · Wirtschaftsinformatik · 1. Semester · Dr. Markus Rosenfelder
Übungsklausur zur Vorbereitung auf die Klausur am Fr, 11.12.2026 – **keine offizielle Musterklausur**

Name, Vorname _____ Kurs _____

Matrikelnummer

Hinweise

- Bearbeitungszeit: **120 Minuten**. Erreichbar sind **120 Punkte** (Faustregel: 1 Punkt ≈ 1 Minute).
- Hilfsmittel: keine (Annahme – bitte für die echte Klausur erfragen). Schreiben Sie mit dokumentenechtem Stift, nicht mit Rot.
- Schreiben Sie Python 3.14 mit Typannotationen an Parametern und Rückgabewerten. Machen Sie Einrückungen deutlich sichtbar (4 Leerzeichen pro Ebene).
- Multiple Choice: Pro Frage ist **genau eine** Antwort richtig. Falsche Kreuze geben 0 Punkte, keine Minuspunkte.
- Verwenden Sie nur Sprachmittel aus VL1–VL5. Eingebaute Funktionen wie `max()` oder `str.count()` sind erlaubt, wenn die Aufgabe sie nicht ausdrücklich verbietet.
- Reicht der Platz nicht, schreiben Sie auf die Rückseite und vermerken Sie das bei der Aufgabe.

Aufgabe	Thema	max. P	erreicht
1	Multiple Choice (VL1–VL5)	12	
2	Ausdrücke auswerten (VL2, VL3)	10	
3	Was gibt dieser Code aus? (VL3, VL4)	10	
4	Trace-Tabelle (VL3)	10	
5	Fehler finden und korrigieren (VL3, VL4)	14	
6	Lückencode: Klasse mit Property (VL5)	10	
7	Funktion schreiben (VL1, VL3, VL4)	12	
8	Rekursion (VL4)	12	
9	Tests mit pytest (VL4)	8	
10	Klassen, Hat-ein-Beziehung, @dataclass (VL5)	14	
11	Konzeptfragen (VL1, VL2, VL4)	8	
	Summe	**120**	

Aufgabe B1: Multiple Choice — 12 P

Kreuzen Sie jeweils **genau eine** richtige Antwort an (je 2 P).

a. Welche Datenstruktur passt zur Zurück-Taste im Browser?

- ☐ Warteschlange (Queue, FIFO)
- ☐ Stapel (Stack, LIFO)
- ☐ Baum
- ☐ Graph

b. Wie lässt sich das Typsystem von Python beschreiben?

- ☐ statisch und schwach typisiert
- ☐ dynamisch, aber stark typisiert
- ☐ dynamisch und schwach typisiert
- ☐ statisch und stark typisiert

c. Was ergibt `0.1 + 0.2 == 0.3`?

- ☐ `True`
- ☐ `False`
- ☐ `TypeError`
- ☐ `0.3`

d. Wie wertet Python `a or b and not c` aus?

- ☐ `(a or b) and (not c)`
- ☐ `a or (b and (not c))`
- ☐ `not (a or b and c)`
- ☐ `(a or b and not) c`

e. Was liefert eine Funktion, deren Rumpf kein `return` enthält?

- ☐ `0`
- ☐ `""`
- ☐ `None`
- ☐ Sie löst einen `SyntaxError` aus.

f. Was erzeugt `@dataclass` aus den annotierten Attributen automatisch?

- ☐ nur `__init__`
- ☐ `__init__`, `__repr__` und `__eq__`
- ☐ Getter und Setter für jedes Attribut
- ☐ `__str__` und eine Property je Attribut

Aufgabe B2: Ausdrücke auswerten — 10 P

Geben Sie für jeden Ausdruck den **Wert** und den **Typ** an (je ½ P).

	Ausdruck	Wert	Typ
a)	`17 // 5`		
b)	`17 % 5`		
c)	`6 / 3`		
d)	`int("42") + 1`		
e)	`float(5)`		
f)	`"Hallo"[-1]`		
g)	`"ll" in "Hallo"`		
h)	`18 <= 17 <= 65`		
i)	`[0] * 3`		
j)	`round(2.675, 2)`		

Aufgabe B3: Was gibt dieser Code aus? 10 P

a) Verschachtelte Schleifen (5 P)

```
matrix = [[1, 2, 3], [4, 5, 6]]
summe = 0
for zeile in matrix:
    for wert in zeile:
        if wert % 2 == 0:
            print(wert, end=" ")
            summe += wert
    print()
print(f"Summe: {summe}")
```

Ausgabe (Leerzeichen am Zeilenende müssen Sie nicht kennzeichnen):

b) `return` in einer Schleife (5 P)

```
def suche(werte: list[int], ziel: int) -> int:
    for i, wert in enumerate(werte):
        if wert == ziel:
            return i
        print(f"prüfe {wert}")
    return -1


def main() -> None:
    print(suche([4, 8, 15, 16], 15))
    print(suche([4, 8], 99))


if __name__ == "__main__":
    main()
```

Ausgabe:

Aufgabe B4: Trace-Tabelle — 10 P

Das Programm berechnet den größten gemeinsamen Teiler (Euklid).

```
1  a, b = 252, 105
2  schritte = 0
3  while b != 0:
4      a, b = b, a % b
5      schritte += 1
6  print(f"ggT = {a} nach {schritte} Schritten")
```

a) Tragen Sie für jeden Durchlauf `a % b` (berechnet vor der Zuweisung in Zeile 4) und die Werte am Ende des Durchlaufs ein. (6 P)

Durchlauf	**`a % b`**	**`a`**	**`b`**	**`schritte`**	**`b != 0?`**
Start	–	252	105	0	

b) Was gibt Zeile 6 aus? (1 P)

c) Jemand ersetzt Zeile 4 durch die zwei Zeilen `a = b` und `b = a % b`. Was gibt das Programm jetzt aus, und warum? (3 P)

Aufgabe B5: Fehler finden und korrigieren **14 P**

Das Programm verwaltet Messwerte. `merke` soll ohne zweites Argument jedes Mal eine **neue** Liste anlegen, `bewerte` soll für jede Temperatur einen Hinweis liefern („heiß“, „Frost“ oder „normal“), `mittelwert` liefert für eine leere Liste 0.0, und `drucke_balken` gibt pro **volle** 10 Einheiten ein `#` aus (35.0 → `###`). Es enthält **genau vier Fehler**.

```
1  def mittelwert(werte: list[float]) -> float:
2      if len(werte) > 0:
3          return sum(werte) / len(werte)
4
5
6  def merke(wert: float, verlauf: list[float] = []) -> list[float]:
7      verlauf.append(wert)
8      return verlauf
9
10
11 def bewerte(temperatur: float) -> str:
12     if temperatur > 30:
13         hinweis = "heiß"
14     elif temperatur < 0:
15         hinweis = "Frost"
16     return hinweis
17
18
19 def drucke_balken(wert: float) -> None:
20     for _ in range(wert / 10):
21         print("#", end="")
22     print()
23
24
25 def main() -> None:
26     print(merke(21.5))
27     print(merke(19.0))  # erwartet: [19.0]
28     print(bewerte(20.0))
29     print(mittelwert([]) + 1)
30     drucke_balken(35.0)
31
32
33 if __name__ == "__main__":
34     main()
```

a) Tragen Sie jeden Fehler ein: Zeile(n), Fehlerart (bei Laufzeitfehlern mit Name der Exception) und Korrektur. (je 3 P)

Nr.	Zeile	Fehlerart	Korrektur (Code)
1			
2			
3			
4			

b) Welche Ausgabe bzw. Fehlermeldung erscheint beim Start des unveränderten Programms **als Erstes** – und warum fällt der Fehler in `merke` nicht durch eine Fehlermeldung auf? (2 P)

Aufgabe B6: Lückencode: Klasse mit Property **10 P**

Ergänzen Sie die Lücken (je 1 P).

```
from decimal import Decimal


class Konto:
    (1) _____ = 0                          # Klassenattribut: Anzahl der Konten

    def __init__(_____(2), inhaber: str, kontostand: Decimal) -> None:
        Konto.anzahl += 1
        self.nummer = f"K{_____(3):03}"
        self.inhaber = inhaber
        self._kontostand = Decimal(0)
        _____(4) = kontostand              # soll den Setter benutzen

    _____(5)
    def kontostand(self) -> Decimal:
        return self._kontostand

    _____(6)
    def kontostand(self, neuer_wert: Decimal) -> None:
        if neuer_wert < 0:
            print("Ungültig: negativer Kontostand.")
            return
        self._kontostand = neuer_wert

    def __str__(self) -> str:
        return _____(7)                    # z. B. "K001 Anna: 100.00 Euro"


def main() -> None:
    anna = Konto("Anna", Decimal("100.00"))
    ben = Konto("Ben", Decimal("-5"))
    print(anna)
    print(ben)
    ben.kontostand = Decimal("20")
    anna.kontostand -= Decimal("150")
    print(anna.kontostand, ben.kontostand)
```

Lücke	Ihr Code
(1)	
(2)	
(3)	
(4)	
(5)	
(6)	
(7)	

Was gibt `main()` aus? (3 P)

Aufgabe B7: Funktion schreiben: Nachbestellung — 12 P

Der Nachbestell-Algorithmus aus VL1: Liegt der Bestand eines Artikels **unter** dem Meldebestand, wird bis zum Höchstbestand aufgefüllt (Bestellmenge = Höchstbestand − Bestand), sonst nicht (Menge 0).

a) Schreiben Sie `nachbestellmengen(bestaende, meldebestaende, hoechstbestaende)` mit Typannotationen (drei Listen von `int`, Rückgabe Liste von `int`) und Docstring im Google-Stil. Beispiel: `nachbestellmengen([12, 45, 8], [20, 30, 25], [100, 150, 120])` → `[88, 0, 112]`. (8 P)

b) Schreiben Sie eine Funktion `main()`, die für die Artikel `["Schrauben M8", "Dichtringe", "Muttern M8"]` mit den Werten aus dem Beispiel nur die tatsächlich nötigen Bestellungen ausgibt, z. B. `Bestellung: Schrauben M8 - 88 Stück`, und so aufgerufen wird, dass sie beim Import der Datei nicht startet. (4 P)

Aufgabe B8: Rekursion — 12 P

a) Rekursive Funktion schreiben (5 P)

Schreiben Sie `umdrehen(text: str) -> str` **rekursiv**, ohne Schleife und ohne `[::-1]`: `umdrehen("abc") == "cba"`, `umdrehen("") == ""`. Tipp: `text[0]` und `text[1:]`.

b) Aufrufstapel (3 P)

Notieren Sie für `umdrehen("abc")` alle Aufrufe (Abstieg) und die Rückgabewerte (Aufstieg).

Ebene	Aufruf	Rückgabewert
1		
2		
3		
4		

c) Fehlender Basisfall (4 P)

```
def summe_liste(zahlen: list[int]) -> int:
    if len(zahlen) == 1:
        return zahlen[0]
    return zahlen[0] + summe_liste(zahlen[1:])
```

Was liefert `summe_liste([4, 1, 5])`? Was passiert bei `summe_liste([])` (mit Name der Exception)? Korrigieren Sie die Abbruchbedingung so, dass die leere Liste 0 ergibt.

Aufgabe B9: Tests mit pytest — 8 P

a) Schreiben Sie für `nachbestellmengen` aus B7 (Datei `lager.py`) eine Testdatei mit drei Testfunktionen. Testen Sie auch die Grenze „Bestand gleich Meldebestand“ und leere Listen. (4 P)

b) Eine Kommilitonin hat `potenz` geschrieben und getestet:

```
def potenz(basis: float, exponent: int) -> float:
    if exponent == 0:
        return 1.0
    return basis + potenz(basis, exponent - 1)
```

```
from potenz import potenz


def test_potenz_null() -> None:
    assert potenz(5, 0) == 1.0


def test_potenz_vier() -> None:
    assert potenz(2, 4) == 16.0


def test_potenz_eins() -> None:
    assert potenz(7, 1) == 7.0
```

```
$ uv run pytest
collected 3 items

test_potenz.py .FF                                                       [100%]

================================== FAILURES ===================================
_____________________________ test_potenz_vier ______________________________

    def test_potenz_vier() -> None:
>       assert potenz(2, 4) == 16.0
E       assert 9.0 == 16.0
E        +  where 9.0 = potenz(2, 4)

test_potenz.py:9: AssertionError
...
======================== 2 failed, 1 passed in 0.01s ==========================
```

(Ausgabe gekürzt, im Format der pytest-Folie aus VL04)

Welcher Test ist bestanden, und warum gerade dieser? Welcher Wert kam tatsächlich heraus, welcher wurde erwartet? Korrigieren Sie `potenz`. (4 P)

Aufgabe B10: Klassen, Hat-ein-Beziehung und `@dataclass` 14 P

a) Klasse `Mitarbeiter` (7 P)

- Klassenattribut `anzahl`; jedes neue Objekt erhält daraus die Personalnummer `M001`, `M002`, … im Attribut `personalnummer`.
- `__init__(self, name: str, monatsgehalt: int)` speichert beide Werte als öffentliche Attribute.
- Nur lesbare Property `jahresgehalt` (12 × Monatsgehalt, ohne eigenes Attribut).
- `gehalt_erhoehen(self, prozent: int) -> bool`: Erlaubt sind 1 bis 20 Prozent. Dann wird das Monatsgehalt auf `monatsgehalt * (100 + prozent) // 100` gesetzt und `True` geliefert; sonst Meldung und `False`.
- `__str__` liefert z. B. `M001 Lena: 4200 Euro/Monat`.

b) Klasse `Abteilung` (4 P)

Eine Abteilung **hat** einen Namen und mehrere Mitarbeiter: `__init__(self, name: str)` legt eine leere, typannotierte Liste `self.mitarbeitende` an; `aufnehmen(self, mitarbeiter: Mitarbeiter) -> None` hängt an; `personalkosten(self) -> int` liefert die Summe aller Jahresgehälter. Ergebnis für Lena (4410 €) und Tim (3800 €): 98520.

c) Datenklasse (3 P)

Schreiben Sie eine `@dataclass Adresse` mit `strasse`, `plz` und `ort` (alle `str`). Was ergeben `a1 == a2` und `a1 is a2` für zwei Adressen mit gleichen Werten – und was ergäbe `==` bei einer normalen Klasse ohne `__eq__`?

Aufgabe B11: Konzeptfragen — 8 P

Antworten Sie jeweils in ein bis zwei Sätzen (je 2 P).

a) Erklären Sie den Unterschied zwischen *Syntax* und *Semantik* und geben Sie zu beiden je ein Fehlerbeispiel.

b) Nennen Sie je einen Vorteil eines Compilers und eines Interpreters.

c) Grenzen Sie imperative und applikative Programmierung voneinander ab.

d) Python kennt keine Methodenüberladung. Was passiert, wenn man zwei Funktionen `addiere` mit unterschiedlich vielen Parametern definiert, und was verwendet man stattdessen?

Aufgabenpool VL1: Einführung: Algorithmen, Datenstrukturen, Python-Werkzeuge

Aufgabe 1.1: Eigenschaften von Algorithmen

★ leicht

Ordnen Sie zu: *terminierend*, *determiniert*, *deterministisch*.

a. Zu jedem Zeitpunkt sind der nächste Schritt und der Zustand eindeutig festgelegt.

b. Bei jeder erlaubten Eingabe endet der Algorithmus nach endlich vielen Schritten.

c. Gleiche Eingaben und Startbedingungen liefern immer das gleiche Ergebnis.

d) Nennen Sie die vier Anforderungen an nützliche Algorithmen aus der Vorlesung.

Aufgabe 1.2: Datenstrukturen wählen

★ leicht

Welche Datenstruktur aus VL1 (Array, Liste, Baum, Graph, Stapel, Warteschlange) passt jeweils am besten? Begründen Sie kurz.

a. Druckaufträge, die in der Reihenfolge ihres Eingangs gedruckt werden

b. „Rückgängig“ in einer Textverarbeitung

c. Organigramm eines Unternehmens

d. Lieferrouten zwischen fünf Lagerstandorten

e. Ablage von Rechnungen nach Jahr, Lieferant und Monat

Aufgabe 1.3: Nachbestell-Algorithmus durchspielen

★★ mittel

Führen Sie den Algorithmus aus VL1 aus: „Ist der Bestand **kleiner** als der Meldebestand, dann Bestellmenge = Höchstbestand − Bestand.“ Welche Bestellungen werden ausgelöst?

Artikel	Bestand	Meldebestand	Höchstbestand	Bestellung?
Kabelbinder	5	10	50	
Dübel	30	30	80	
Schrauben M8	12	20	100	
Unterlegscheiben	0	40	200	
Muttern M8	41	25	120	

Aufgabe 1.4: Einen Algorithmus aufschreiben

★★ mittel

Beschreiben Sie in nummerierten Schritten, wie ein Urlaubsantrag geprüft wird (Resturlaub, Vertretung, Sperrzeiten). Prüfen Sie Ihren Algorithmus an den vier Anforderungen. Welcher Schritt ist bei „Antrag kurz ansehen“ problematisch?

Aufgabe 1.5: Vom Quellcode zum Programm

★★ mittel

a) Ergänzen Sie die Tabelle.

Strategie	Ablauf	Vorteil	Nachteil	Beispiel
Interpreter				
Compiler				
Zwischencode				

b) Zu welcher Strategie gehört Python (CPython)? Was bedeutet JIT?

Aufgabe 1.6: Syntax und Werkzeuge

★★★ anspruchsvoll

a) Welche Fehlermeldung erscheint beim Start dieses Programms, und welche erscheint, nachdem Sie diesen Fehler behoben haben?

```
1  # Begruessung
2  Print("Hallo DHBW")
3  if True:
4  print("eingerückt?")
```

b) Was bewirken `pwd`, `cd ..`, `uv run hallo.py`, `uv run ruff check .` und `uv run ty check`?

Aufgabenpool VL2: Variablen, Datentypen und Operatoren

Aufgabe 2.1: Datentypen

★ leicht

Was gibt das Programm aus?

```
print(type(42))
print(type(3.0))
print(type("42"))
print(type(True))
print(type(None))
wert = 3,5
print(wert, type(wert))
```

Aufgabe 2.2: Namen und Konventionen

★ leicht

Ist der Name gültig? Wenn ja: Wofür verwendet man ihn nach Python-Konvention (Variable/Funktion, Konstante, Klasse) – oder verstößt er gegen die Konvention?

Name	gültig?	Konvention / Verwendung
`kunden_anzahl`		
`2tes_konto`		
`MWST_SATZ`		
`kundenAnzahl`		
`netto-preis`		
`class`		
`groesse`		
`Kunde`		

Aufgabe 2.3: Zuweisungen nachvollziehen

★★ mittel

Was gibt das Programm aus? Notieren Sie Zwischenwerte.

```
x = 5
x *= 3
x -= 1
x //= 4
x **= 2
print(x)
a, b = 1, 2
a, b = b, a + b
a, b = b, a + b
print(a, b)
y = 7
++y
print(y)
```

Aufgabe 2.4: Euro in Cent

★★ mittel

Was gibt das Programm aus? Erklären Sie die zweite Zeile und nennen Sie zwei Abhilfen.

```
from decimal import Decimal

print(int(9.99 * 100))
print(int(19.99 * 100))
print(19.99 * 100)
print(round(19.99 * 100))
print(int(Decimal("19.99") * 100))
```

Aufgabe 2.5: f-Strings und Umwandlungen

★★ mittel

Was gibt das Programm aus?

```
name = "Ada"
alter = 20
preis = 7.5
netto = 49.9
print(f"{preis:.2f} €")
print(f"{name} ist {alter + 1} Jahre alt")
print(f"Brutto: {netto * 1.19:.2f}")
print(f"{name * 2}|{len(name)}|{name[0]}{name[-1]}")
print(int(-3.7), float(5), str(42) + "1")
```

Aufgabe 2.6: Fehlerarten erkennen

★★ mittel

Welche Fehlerart liegt vor (Syntaxfehler, Typfehler, Laufzeitfehler, Logikfehler)? Wann wird er bemerkt? Wie lautet die letzte Zeile der Fehlermeldung ungefähr?

a)

```
zahl: int = 10
print(f"Die Zahl ist {zahl}"
```

b)

```
alter = 36
print("Alter: " + alter)
```

c)

```
a = 10
b = 0
print(a / b)
```

d)

```
anzahl: int
print(anzahl)
```

e)

```
print(int("12.5"))
```

f)

```
x = 5
x++
```

Aufgabe 2.7: Mini-Rechner und Jahreszahl ohne `int()`

★★★ anspruchsvoll

a) Gegeben `a = "12"` und `b = "3"`. Geben Sie genau `a=12, b=3 | sum=15, diff=9, prod=36, quot=4.0` aus. Was ergeben `a + b` und `a * 3`?

b) Wandeln Sie `jahr = "2026"` **ohne** `int()` in die Zahl 2026 um. Tipp: `ord("7") - ord("0")` ergibt 7.

Aufgabenpool VL3: Kontrollstrukturen und Listen

Aufgabe 3.1: Noten mit `if/elif`

★ leicht

Ab 90 Punkten „sehr gut“, ab 75 „gut“, ab 60 „befriedigend“, ab 50 „ausreichend“, sonst „nicht bestanden“. Was gibt der Code aus, warum ist er falsch? Schreiben Sie ihn richtig und nennen Sie sinnvolle Testwerte.

```
def note(punkte: int) -> str:
    if punkte >= 50:
        return "ausreichend"
    elif punkte >= 75:
        return "gut"
    elif punkte >= 90:
        return "sehr gut"
    return "nicht bestanden"


print(note(95))
```

Aufgabe 3.2: Kurzschlussauswertung

★ leicht

Es gilt `x = 0`. Welche Ausdrücke liefern einen Wert, welche einen `ZeroDivisionError`?

a. `x != 0 and 10 / x > 1`

b. `10 / x > 1 and x != 0`

c. `x == 0 or 10 / x > 1`

d. `x > 0 or 10 / x > 1`

Aufgabe 3.3: Wahrheitswerte und bedingter Ausdruck

★★ mittel

Was gibt das Programm aus?

```
for wert in [[], "0", 0, " ", None, [0], -1, ""]:
    print("ja" if wert else "nein", end=" ")
print()
a, b, c = False, True, False
print(a or b and not c)
print(not a and c)
print(a != b)
```

Aufgabe 3.4: Schleifen: `range`, `break`, `continue`

★★ mittel

Was gibt das Programm aus?

```
for i in range(5, 0, -2):
    print(i, end=" ")
print()
for i in range(1, 20, 4):
    print(i, end=" ")
print()
for i in range(10, 3):
    print(i, end=" ")
print("leer?")
for i in range(1, 11):
    if i % 3 == 0:
        continue
    if i > 8:
        break
    print(i, end=" ")
print()
n = 10
while n > 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
    print(n, end=" ")
print()
for i in range(3):
    for j in range(i):
        print(f"{i}{j}", end=" ")
print()
```

Aufgabe 3.5: Parkgebühren mit `match`

★★ mittel

Was geben die fünf Aufrufe aus? Rechnen Sie nach.

```
from typing import Final

GRUNDGEBUEHR: Final = 1.50
HOECHSTPREIS: Final = 50.0


def parkgebuehr(typ: str, stunden: int) -> None:
    gueltig = True
    kosten = 0.0
    match typ:
        case "PKW":
            kosten = stunden * 2.0
            if stunden >= 10:
                kosten *= 0.90
        case "Motorrad":
            kosten = stunden * 1.0
        case "Transporter":
            kosten = stunden * 3.50
            if stunden >= 5:
                kosten *= 0.80
        case _:
            print(f"Unbekannter Fahrzeugtyp: {typ}")
            gueltig = False
    if gueltig:
        kosten += GRUNDGEBUEHR
        if kosten > HOECHSTPREIS:
            kosten = HOECHSTPREIS
        print(f"{typ}, {stunden} h: {kosten:.2f} Euro")


parkgebuehr("Transporter", 6)
parkgebuehr("PKW", 12)
parkgebuehr("PKW", 9)
parkgebuehr("Motorrad", 60)
parkgebuehr("LKW", 2)
```

Aufgabe 3.6: Schaltjahr

★★ mittel

Schreiben Sie `ist_schaltjahr(jahr: int) -> bool` mit einem einzigen logischen Ausdruck: durch 4 teilbar, aber nicht durch 100 – außer durch 400. Welche Ergebnisse erwarten Sie für 1900, 2000, 2024, 2026?

Aufgabe 3.7: Temperaturliste

★★★ anspruchsvoll

Gegeben `wochentage = ["Mo", "Di", "Mi", "Do", "Fr"]` und `temperaturen = [-3.5, -1.0, -4.5, -2.0, -0.5]`. Geben Sie jeden Tag mit Temperatur aus und berechnen Sie Durchschnitt und Maximum mit einer Schleife (ohne `sum`/`max`), danach mit eingebauten Funktionen. Warum darf das Maximum nicht mit 0 starten?

Aufgabe 3.8: Listenfalle und Sieb des Eratosthenes

★★★ anspruchsvoll

a) Was gibt der Code aus? Erklären Sie die erste Zeile.

```
falsch = [[0] * 3] * 2
falsch[0][0] = 1
print(falsch)

richtig = []
for _ in range(2):
    richtig.append([0] * 3)
richtig[0][0] = 1
print(richtig)

noten = [2.3, 1.0, 3.7, 1.7]
print(noten[1:3], noten[-1], sorted(noten), noten)
```

b) Geben Sie mit dem Sieb des Eratosthenes alle Primzahlen bis 30 aus. Verwenden Sie eine `bool`-Liste `ist_prim`.

Aufgabenpool VL4: Funktionen, Rekursion und Tests

Aufgabe 4.1: Standardwerte und Schlüsselwortargumente

★ leicht

a) Was geben die Aufrufe aus? Was passiert in der letzten Zeile?

```
def brutto(netto: float, satz: float = 0.19) -> float:
    return round(netto * (1 + satz), 2)


print(brutto(100))
print(brutto(100, 0.07))
print(brutto(satz=0.07, netto=100))
print(brutto(19.99))
print(brutto())
```

b) Warum lässt sich diese Funktion gar nicht erst definieren?

```
def f(a: int = 1, b: int) -> int:
    return a + b
```

Aufgabe 4.2: Gültigkeitsbereiche

★★ mittel

Was geben die drei Programme aus (bzw. welcher Fehler)?

```
def f() -> None:
    y = 3
    print("in f:", y)


f()
print(y)
```

```
def erhoehe(wert: int) -> None:
    wert = wert + 1
    print(f"In der Funktion: {wert}")


zahl = 5
erhoehe(zahl)
print(f"Nach dem Aufruf: {zahl}")
```

```
MWST_SATZ = 0.19
zaehler = 0


def preis(netto: float) -> float:
    return netto * (1 + MWST_SATZ)


def erhoehe() -> None:
    zaehler += 1


print(preis(100.0))
erhoehe()
```

Aufgabe 4.3: `break` oder `return`?

★★ mittel

Was gibt das Programm aus?

```
def mit_break() -> None:
    for i in range(1, 11):
        if i == 4:
            break
        print(i, end=" ")
    print("Schleife beendet.")


def mit_return() -> None:
    for i in range(1, 11):
        if i == 4:
            return
        print(i, end=" ")
    print("Schleife beendet.")


mit_break()
mit_return()
print("| fertig")
```

Aufgabe 4.4: Veränderbarer Standardwert

★★ mittel

Was gibt das Programm aus? Korrigieren Sie `anhaengen`.

```
def anhaengen(wert: int, liste: list[int] = []) -> list[int]:
    liste.append(wert)
    return liste


a = anhaengen(1)
b = anhaengen(2)
c = anhaengen(3, [0])
print(a, b, c)
print(a is b)
```

Aufgabe 4.5: Tupel als Rückgabe und `*args`

★★ mittel

a) Was gibt das Programm aus?

```
def min_max(zahlen: list[int]) -> tuple[int, int]:
    return min(zahlen), max(zahlen)


def summe(*zahlen: int) -> int:
    ergebnis = 0
    for zahl in zahlen:
        ergebnis += zahl
    return ergebnis


kleinste, groesste = min_max([4, 9, 2, 7])
print(kleinste, groesste)
print(min_max([5]))
print(summe(1, 2, 3, 4), summe())
```

b) Schreiben Sie `durchschnitt(*werte: float) -> float` mit Docstring; ohne Argumente soll 0.0 herauskommen.

Aufgabe 4.6: Rekursion nachvollziehen

★★ mittel

Was gibt das Programm aus? Zeichnen Sie den Aufrufstapel für `summe_bis(4)` (Abstieg und Aufstieg). Warum erscheinen die Zahlen bei `raetsel` aufsteigend?

```
def summe_bis(n: int) -> int:
    return 0 if n == 0 else n + summe_bis(n - 1)


def raetsel(n: int) -> None:
    if n > 0:
        raetsel(n - 1)
        print(n, end=" ")


print(summe_bis(4))
raetsel(4)
print()
```

Aufgabe 4.7: Rekursive Funktionen schreiben

★★★ anspruchsvoll

Schreiben Sie jeweils rekursiv, mit Typannotationen und einzeiligem Docstring:

a. `zaehle_zeichen(text, zeichen) -> int` ohne `text.count()`; `("banana", "a")` → 3

b. `potenz(basis: float, exponent: int) -> float` ohne `**`, auch für negative Exponenten; `potenz(2, -2)` → 0.25

c. `maximum(zahlen: list[int], index: int = 0) -> int` ohne `max()`; von außen ohne `index` aufrufbar

Aufgabe 4.8: Applikativ programmieren und testen

★★★ anspruchsvoll

a) Setzen Sie die Funktion `h(p, q, x, y) = if p ∧ q then 4x + 2y else 5x − 3y fi` als Python-Funktion mit einem bedingten Ausdruck um.

b) Schreiben Sie `fibonacci(n: int) -> int` rekursiv (`fibonacci(0) == 0`, `fibonacci(1) == 1`).

c) Schreiben Sie eine Testdatei für pytest mit mindestens vier Tests (beide Zweige von `h`, Randfälle von `fibonacci`).

d) Warum ist die naive Fibonacci-Rekursion langsam?

Aufgabenpool VL5: Objektorientierte Programmierung I

Aufgabe 5.1: Grundbegriffe und UML

★ leicht

a) Erklären Sie Klasse, Objekt (Instanz), Attribut und Methode am Beispiel eines Mitarbeiters. Zeichnen Sie ein UML-Klassendiagramm mit zwei Attributen und zwei Methoden.

b) Was passiert bei diesem Code, und warum?

```
class Produkt:
    def __init__(name: str, preis: float) -> None:
        self.name = name


kaffee = Produkt("Kaffee", 12.99)
```

Aufgabe 5.2: Methoden und `__str__`/`__repr__`

★ leicht

Was gibt `main()` aus?

```
class Konto:
    def __init__(self, inhaber: str, kontostand: int = 0) -> None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    def einzahlen(self, betrag: int) -> None:
        self.kontostand += betrag

    def abheben(self, betrag: int) -> bool:
        if betrag > self.kontostand:
            print("Abgelehnt: Der Kontostand reicht nicht.")
            return False
        self.kontostand -= betrag
        return True

    def __str__(self) -> str:
        return f"Konto von {self.inhaber}: {self.kontostand} Euro"

    def __repr__(self) -> str:
        return f"Konto({self.inhaber!r}, {self.kontostand})"


def main() -> None:
    anna = Konto("Anna", 100)
    ben = Konto("Ben")
    anna.einzahlen(50)
    print(anna.abheben(500))
    print(anna.abheben(30))
    Konto.einzahlen(ben, 25)
    print(anna)
    print([anna, ben])


if __name__ == "__main__":
    main()
```

Aufgabe 5.3: Namen, Identität und Gleichheit

★★ mittel

Was gibt `main()` aus?

```
from dataclasses import dataclass


class Konto:
    def __init__(self, inhaber: str, kontostand: int = 0) -> None:
        self.inhaber = inhaber
        self.kontostand = kontostand

    def einzahlen(self, betrag: int) -> None:
        self.kontostand += betrag


@dataclass
class Kunde:
    name: str
    ort: str


def main() -> None:
    konto1 = Konto("Clara", 100)
    konto2 = konto1
    konto2.einzahlen(50)
    konto3 = Konto("Clara", 150)
    print(konto1.kontostand)
    print(konto1 is konto2, konto1 == konto3)
    konto2 = Konto("Clara")
    print(konto1.kontostand, konto2.kontostand)
    k1 = Kunde("Anna Berger", "Heilbronn")
    k2 = Kunde("Anna Berger", "Heilbronn")
    print(k1 == k2, k1 is k2)
    print(k1)


if __name__ == "__main__":
    main()
```

Aufgabe 5.4: Klassenattribute und Tippfehler

★★ mittel

Was gibt das Programm aus? Was würde `ty check` zur vorletzten Zeile sagen?

```
class Konto:
    anzahl_konten = 0

    def __init__(self, inhaber: str) -> None:
        Konto.anzahl_konten += 1
        self.nummer = f"K{Konto.anzahl_konten:03}"
        self.inhaber = inhaber
        self.kontostand = 0


anna = Konto("Anna")
ben = Konto("Ben")
print(anna.nummer, ben.nummer)
ben.anzahl_konten += 1
print(Konto.anzahl_konten, anna.anzahl_konten, ben.anzahl_konten)
Konto.anzahl_konten += 10
print(Konto.anzahl_konten, anna.anzahl_konten, ben.anzahl_konten)
anna.kontostnad = 500
print(anna.kontostand)
```

Aufgabe 5.5: Klasse `Produkt` schreiben

★★ mittel

Schreiben Sie eine Klasse `Produkt` mit `name: str`, `preis: Decimal`, `bestand: int` und den Methoden `lagerwert() -> Decimal`, `einlagern(menge) -> None` (nicht positive Mengen: Meldung, keine Änderung), `verkaufen(menge) -> bool` (zu wenig Bestand: Meldung, `False`), `__str__` (`Kaffee 1 kg: 12.99 Euro, Bestand 40`) und `__repr__`. Testen Sie in `main()`: 5 verkaufen, 100 verkaufen, 10 einlagern, −3 einlagern, Produkt und Lagerwert ausgeben.

Aufgabe 5.6: Kapselung mit Properties

★★★ anspruchsvoll

Ändern Sie `Produkt`: Bestand in `_bestand` mit nur lesbarer Property `bestand`; Preis in `_preis` mit Property und Setter, der negative Preise mit Meldung abweist (auch `__init__` nutzt den Setter); berechnete Property `bruttopreis` (19 % MwSt., `MWST_SATZ: Final = Decimal("0.19")`, gerundet mit `.quantize(Decimal("0.01"))`). Was passiert bei `kaffee.bestand = 100`?

Aufgabe 5.7: Warenkorb: Hat-ein-Beziehung und `@dataclass`

★★★ anspruchsvoll

Schreiben Sie `@dataclass Kunde` (`name`, `email`), `@dataclass Position` (`produkt: Produkt`, `menge: int`, Methode `summe()`) und eine normale Klasse `Warenkorb`, die einen Kunden und eine Liste von Positionen **hat**: `hinzufuegen(produkt, menge)` (nur, wenn der Bestand reicht), `gesamtsumme() -> Decimal` und `ausgeben()`. Testen Sie mit Kaffee (12.99 €, Bestand 40) und Tee (3.49 €, Bestand 120): 2 × Kaffee, 3 × Tee, 50 × Kaffee. Warum ist `Warenkorb` keine Datenklasse?